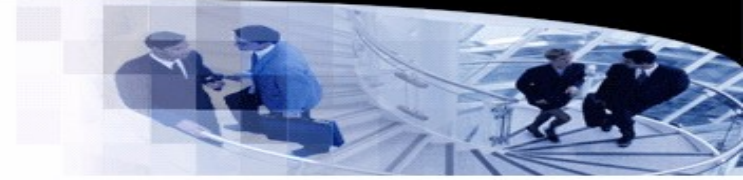




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

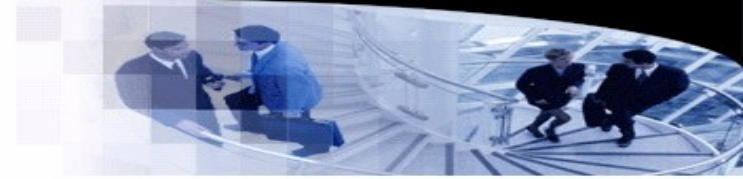
Lecture 5. Regular Languages

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Introduction



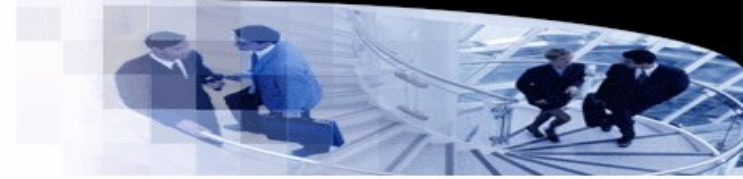
- Regular languages

- One of the most simple formal languages.
- For programming languages, the set of words (tokens) can be categorized as regular languages.

- Models

- Finite state automata
- Regular expressions
- Regular grammars

Automata

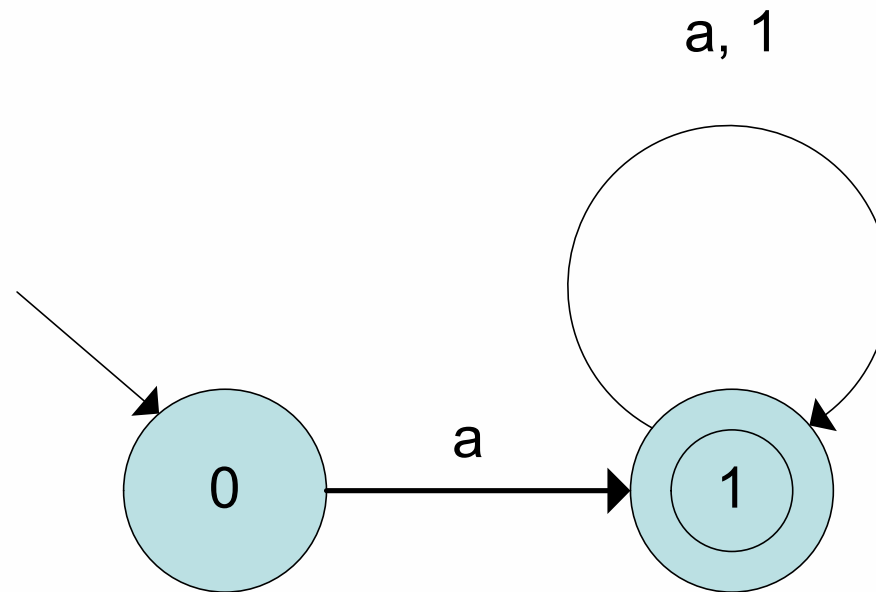


- A program that recognizes all identifiers.

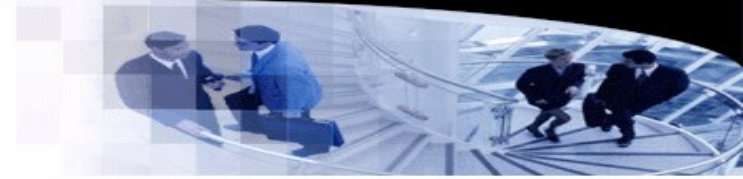
```
int nState = 0;
while (ch = getch()) {
    if (nState == 0)
        if (ch == 'a')
            nState = 1;
        else
            return ERROR;
    else
        if (ch != 'a' && ch != '1')
            return ERROR;
}
return 0;
```

Automata

- The program can be represented in a graphical form.



Automata

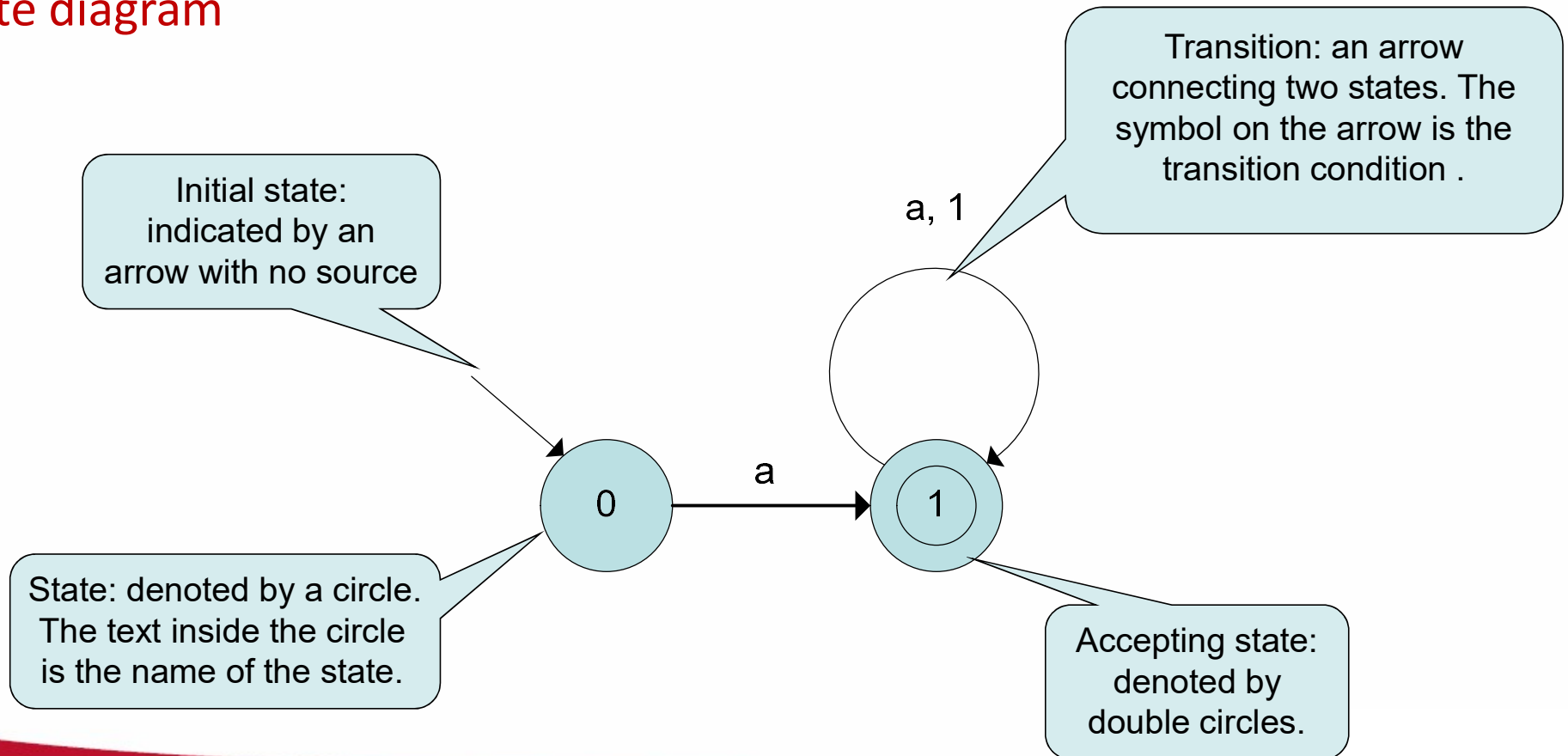
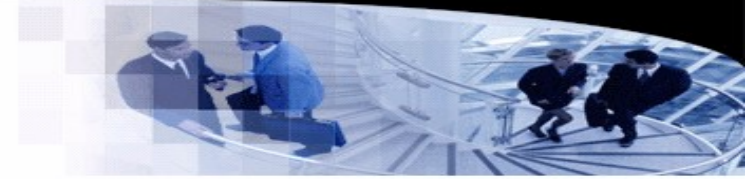


- Finite state automata (FA)

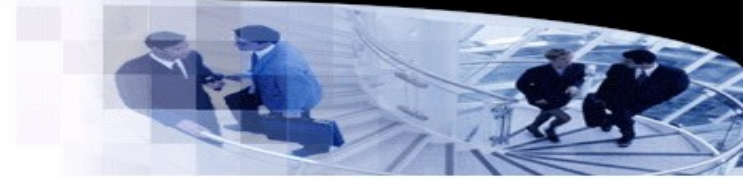
- A computing device that has only one variable.
- The range of the variable is finite.
- Each possible value of the variable is called a **state**.
- The device starts from an **initial state**.
- The device reads symbols one by one from the **input**.
- Every time a symbol is read, the device updates its state according by some rules.
- Once all input symbols are read, the device stops and checks the state.
- If the state is one of the accepting states, the device **accepts** the input.
- The device **rejects** the input otherwise.

Automata

- State diagram



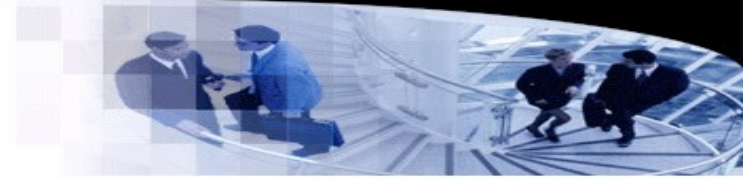
Automata



- Syntax of finite automata

- An automaton is a tuple $M = (Q, \Sigma, \delta, q_0, F)$, where
 - Q is a finite set of states,
 - Σ is a set of symbols (alphabet),
 - $\delta: Q \times \Sigma \rightarrow Q$ is the transition function,
 - q_0 is the initial state,
 - and $F \subseteq Q$ is a set of accepting (final) states.
- Example: $M = (Q, \Sigma, \delta, q_0, F)$, where
 - $Q = \{0, 1\}$,
 - $\Sigma = \{a, 1, \$\}$,
 - $\delta = \{((0, a), 1), ((1, a), 1), ((1, 1), 1)\}$,
 - and $F = \{1\}$.

Automata



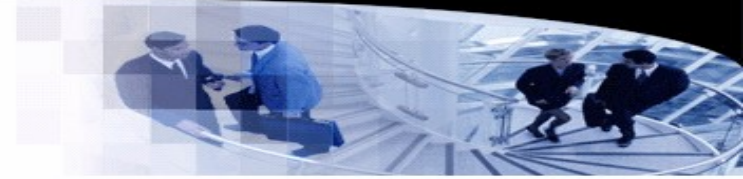
- **Language of finite automata**

- Given an automaton $M = (Q, \Sigma, \delta, q_0, F)$, a **run** of M is a sequence of states $\rho = s_0 s_1 \dots s_n$ such that
 - $s_0 = q_0$,
 - $s_n \in F$,
 - and $\forall 0 \leq i < n (\exists a \in \Sigma (\delta(s_i, a) = s_{i+1}))$.
- A string $\alpha = a_1 a_2 \dots a_n$ is **accepted** by M , if there is a run $\rho = s_0 s_1 \dots s_n$ such that
 - $\forall 0 \leq i < n (\delta(s_i, a_{i+1}) = s_{i+1})$.
- The language of M , which is denoted as $\mathcal{L}(M)$, is the set of all accepted strings.

- **Regular languages**

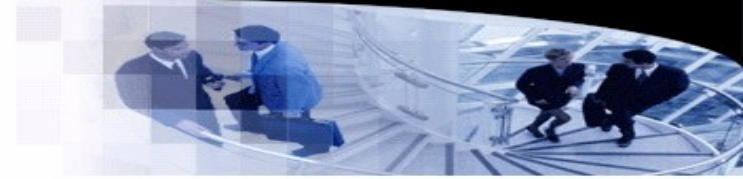
- An automaton **recognizes** a language, if the automaton accepts exactly all strings in the language.
- A language is **regular**, if there is an automaton that recognizes the language.

Automata



- Exercises: design automata to recognize the following languages ($\Sigma = \{0,1\}$)
 - $L_1 = \{ \omega \mid \omega \text{ starts with "101".} \}$
 - $L_2 = \{ \omega \mid \omega \text{ contains the substring "01".} \}$
 - $L_3 = \{ \omega \mid \omega \text{ is a valid octal number.} \}$
 - $L_4 = \{ \omega \mid \omega \text{ is a valid hexadecimal number.} \}$
 - $L_3 \cup L_4$

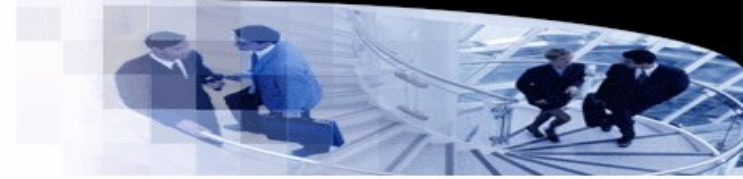
Automata



- **Composition of automata**

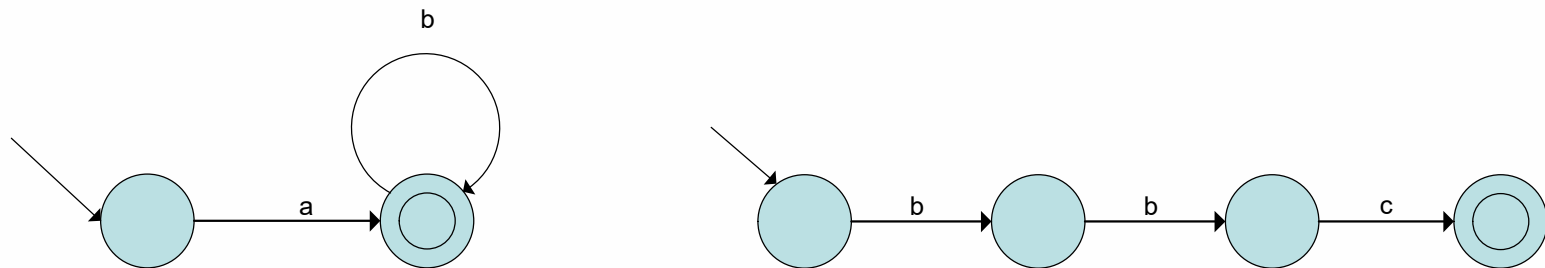
- **Theorem:** If L_1 and L_2 are regular, so is $L_1 \cup L_2$.
- Proof idea:
 - Let M_1 and M_2 be the automata that recognize L_1 and L_2 , respectively.
 - Construct an automaton M , in which each state is an ordered pair (s_1, s_2) , where s_1 simulates the state of M_1 and s_2 simulates the state of M_2 .
 - Every time a symbol is read, update s_1 and s_2 according to the transition rules of M_1 and M_2 , respectively.
 - Once the input is ended, M accepts the input if either s_1 or s_2 is in the accepting set of M_1 and M_2 .
- The theorem implies that we can construct large automata to recognize complicated language by composing simple automata.

Automata



- Composition of automata

- **Theorem:** If L_1 and L_2 are regular, so is L_1L_2 .
 - Can we construct an automaton to recognize L_1L_2 ?
 - Consider the following automate.

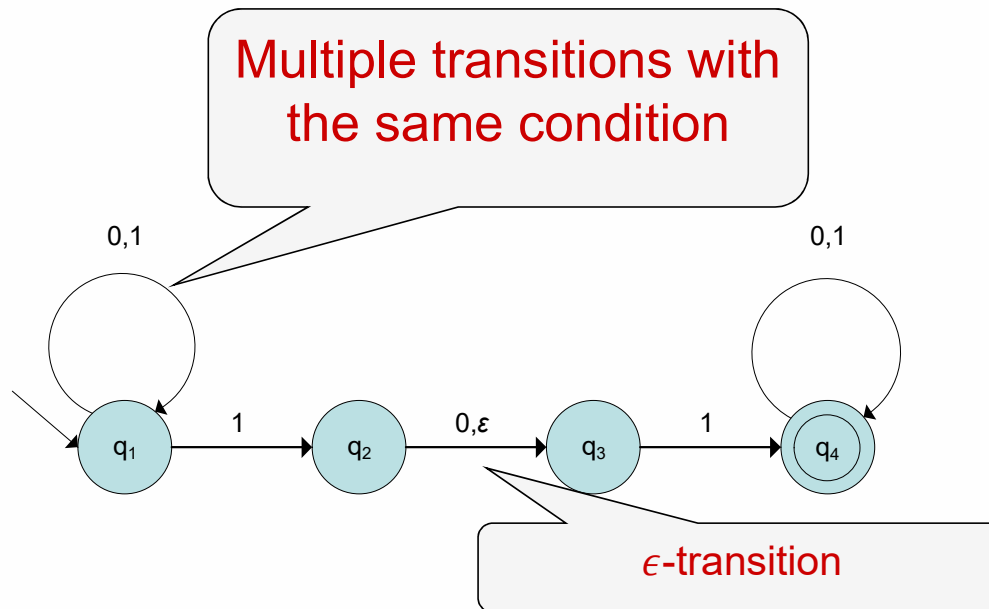


- In this case, we need a mechanism to guess all possibilities.
- This mechanism is called nondeterministic.

Automata

- Nondeterministic automata

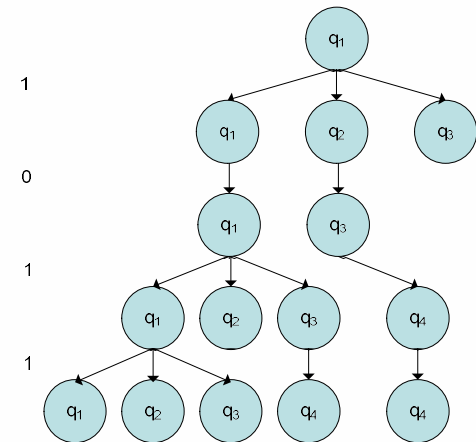
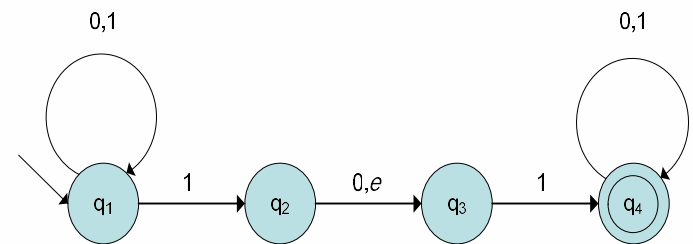
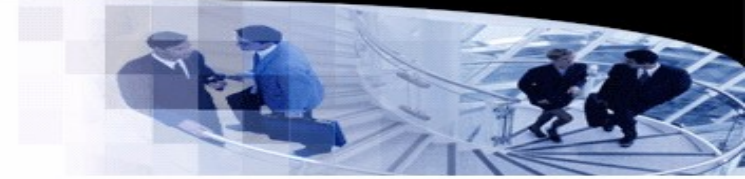
- Nondeterministic automata (NFA)
- Deterministic automata (DFA)



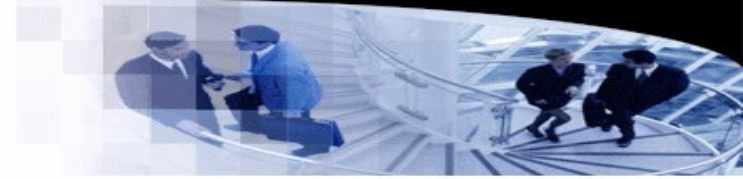
Automata

- **Nondeterministic automata**

- Whenever there are multiple transitions that satisfies the transition condition, the machine forks to multiple copies to deal with multiple choices.
- When the input is end, the machine accepts the input if there is a copy that accepts the input.
- Given the input 1011, the running of the automaton is depicted on the right-hand side.



Automata



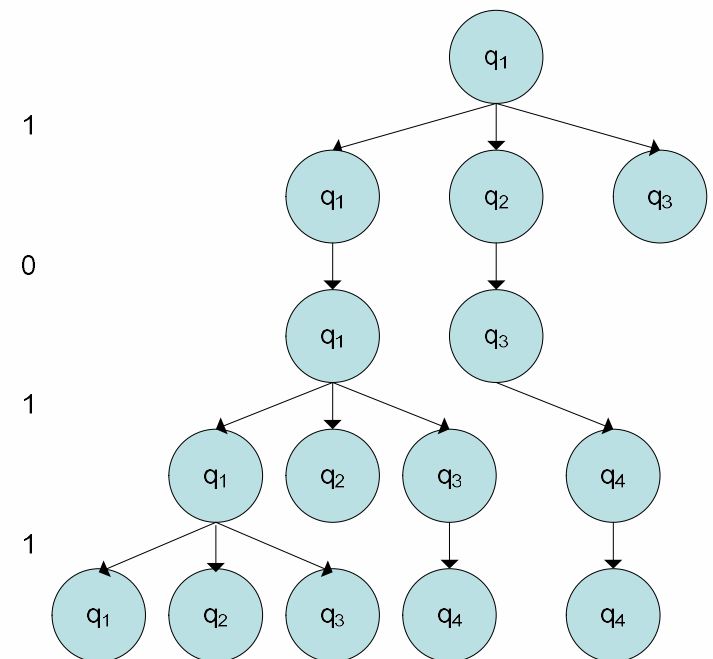
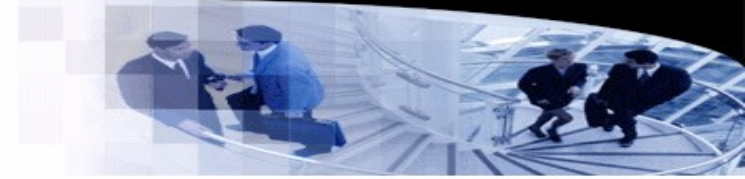
- **Nondeterministic automata**

- A nondeterministic automata (NFA) is a tuple $M = (Q, \Sigma, \delta, q_0, F)$, where
 - Q is a finite set of states,
 - Σ is a set of symbols (alphabet),
 - $\delta: Q \times \Sigma_{\epsilon} \rightarrow 2^Q$ is the transition function where $\Sigma_{\epsilon} = \Sigma \cup \{\epsilon\}$,
 - q_0 is the initial state,
 - and $F \subseteq Q$ is a set of accepting (final) states.

Automata

- Equivalence between NFA and DFA

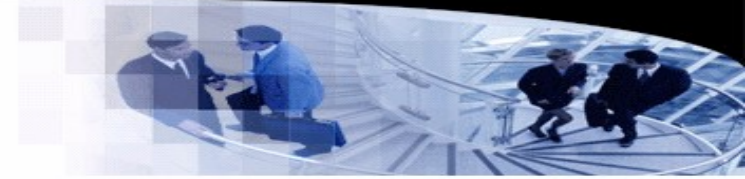
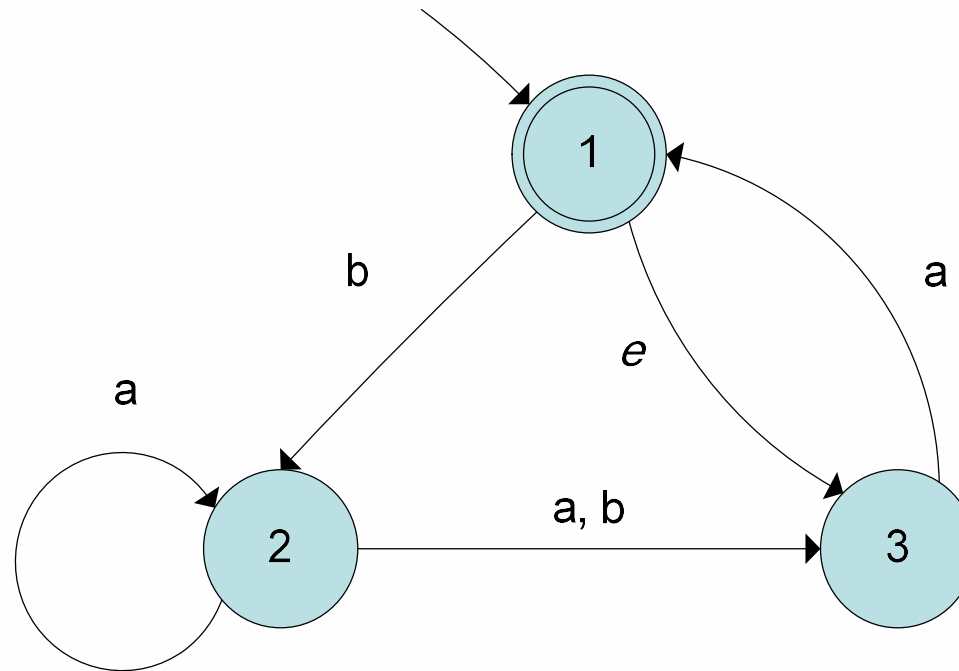
- Consider the right-hand-side NFA.
- We can construct a DFA where each state is a set.
- Each element of the set is a state of the NFA.
- The set can be used to simulate copies of machines during the run of the NFA.
- Observation: given a set of states and an input symbol, the calculation of the target set of states is deterministic.
- Hence, we can transform an NFA into an equivalent DFA.



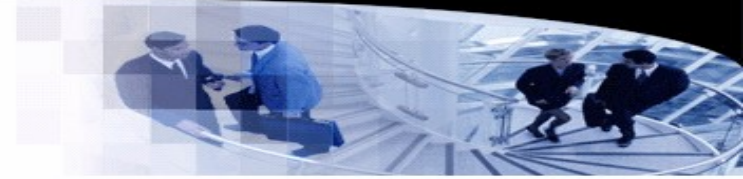
Automata

- Example

- Transform the following NFA into a DFA.



Regular expression



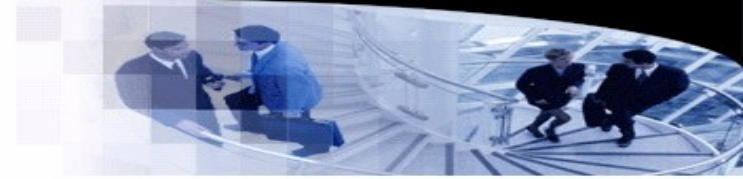
- Regular operators

- We can use regular operators to construct complex expressions, so as to define complex languages.

- Syntax of regular expressions

- R is called a regular expressions, if
 - $R = a$ where $a \in \Sigma$,
 - $R = \epsilon$,
 - $R = \emptyset$,
 - $R = R_1 + R_2$ (or $R_1 | R_2$ or $R_1 \cup R_2$) where R_1 and R_2 are regular expressions,
 - $R = R_1 R_2$ (or $R_1 \cdot R_2$) where R_1 and R_2 are regular expressions,
 - or $R = R_1^*$ where R_1 is a regular expression.

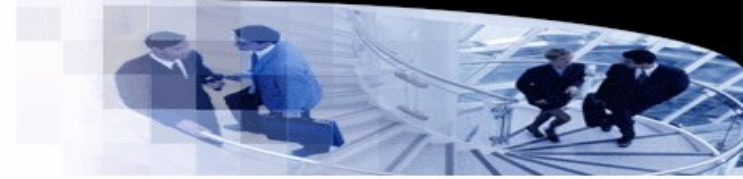
Regular expression



- Semantics of regular expressions

- Let R be a regular expressions. $\mathcal{L}(R)$, the language of R is defined by the following rules.
 - $\mathcal{L}(a) = \{a\}$
 - $\mathcal{L}(\epsilon) = \{\epsilon\}$
 - $\mathcal{L}(\emptyset) = \emptyset$
 - $\mathcal{L}(R_1 + R_2) = \mathcal{L}(R_1) \cup \mathcal{L}(R_2)$
 - $\mathcal{L}(R_1 \cdot R_2) = \mathcal{L}(R_1) \cdot \mathcal{L}(R_2)$
 - $\mathcal{L}(R_1^*) = \mathcal{L}(R_1)^*$

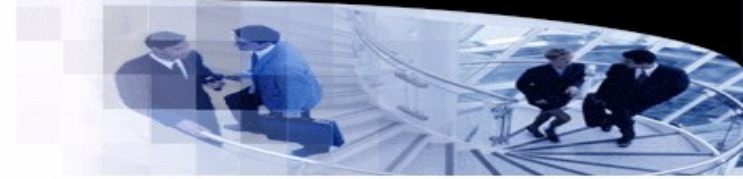
Regular expression



- Equivalence between regular expressions and automata

- A language defined by a regular expression is regular.
- **Proof idea:**
 - For each regular expression, we can construct an equivalent NFA.
 - The automaton is constructed by induction.
 - Base cases:
 - $R = a$
 - $R = \epsilon$
 - $R = \emptyset$
 - Inductive steps:
 - Thompson's construction.

Regular expression

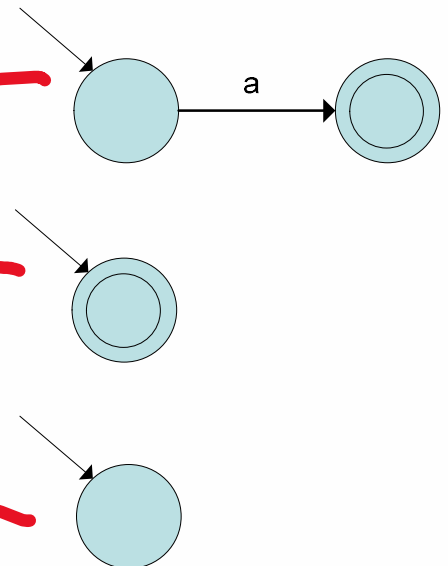


- Equivalence between regular expressions and automata

- A language defined by a regular expression is regular.

- **Proof idea:**

- For each regular expression, we can construct an equivalent NFA.
- The automaton is constructed by induction.
- Base cases:
 - $R = a$
 - $R = \epsilon$
 - $R = \emptyset$
- Inductive steps:
 - Thompson's construction.

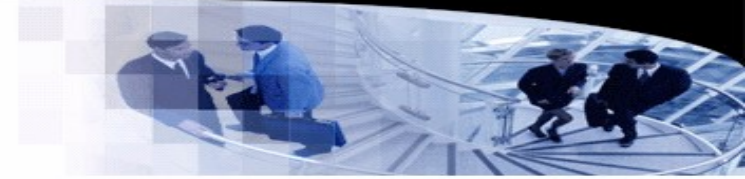
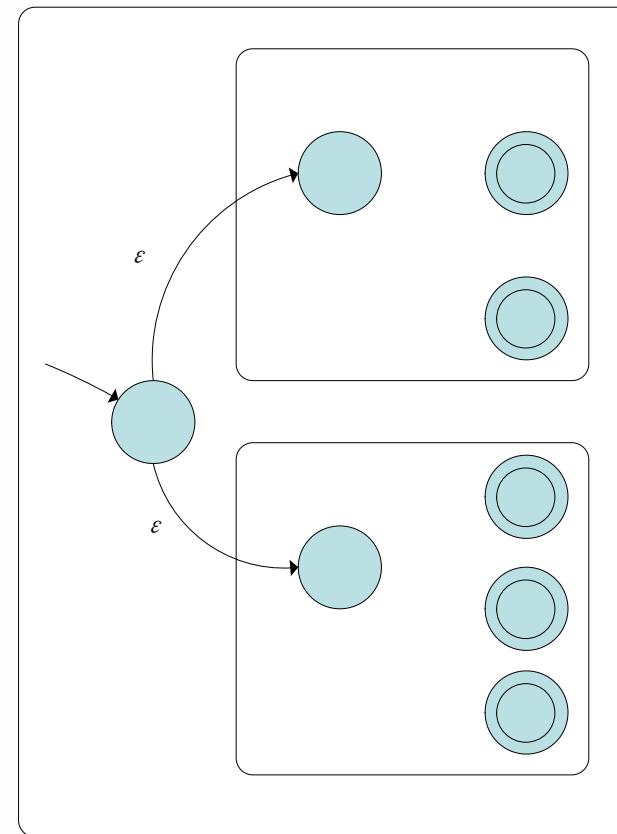
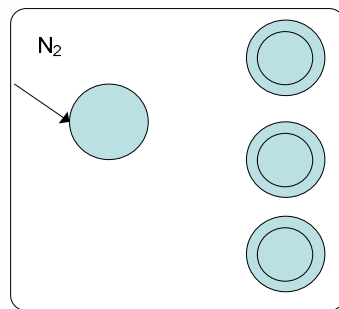
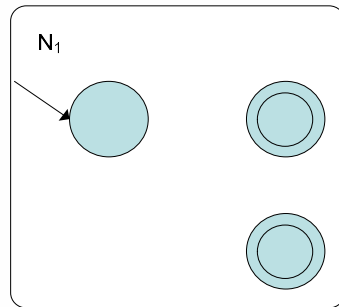


Regular expression

- Equivalence between regular expressions and automata

- Thompson's construction.

- $R_1 + R_2$

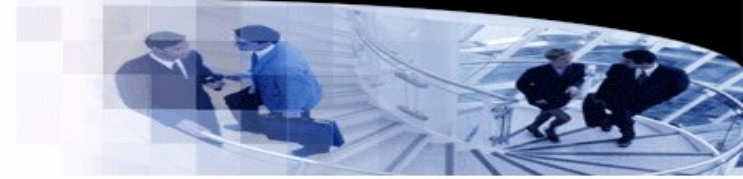
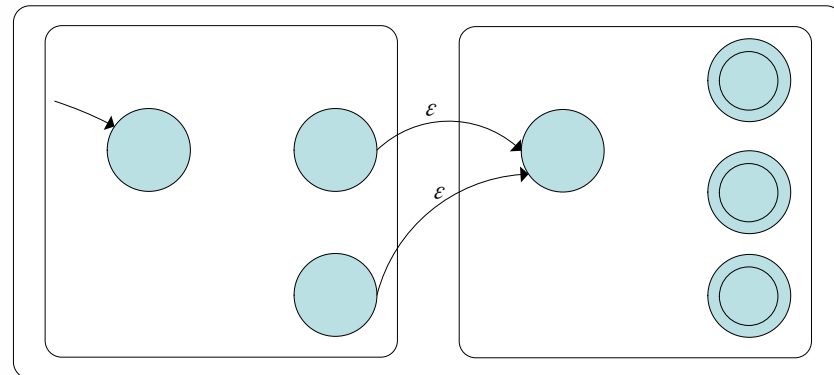
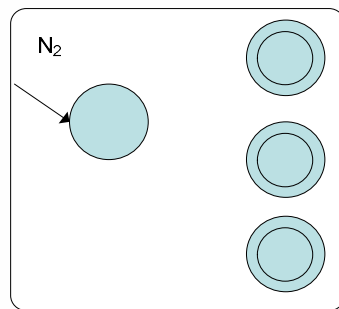
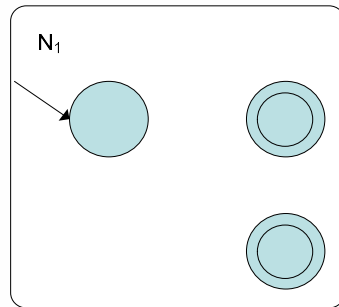


Regular expression

- Equivalence between regular expressions and automata

- Thompson's construction.

- $R_1 \cdot R_2$



Regular expression

- Equivalence between regular expressions and automata
 - Thompson's construction.
 - R_1^*

